Linux Beginner Guide

Jaewoong Lee

Ulsan National Institute of Science and Technology

jaewoong@unist.ac.kr

June 13, 2026

Introduction

In this guide, I assume that you can open a Linux shell in one of these ways:

1. A current Ubuntu LTS desktop, WSL, or course-provided Linux server
2. A terminal running bash or zsh
3. A basic terminal editor such as Vim, Neovim, or nano
4. A registered SSH public key if the lab uses a remote server

With this guide, you can use and understand a Linux system.
The commands are written for a beginner lab in 2026. Your prompt, user name, host name, or Linux distribution may look different, and that is fine.

Overview

Linux?

Figure: Linus Torvalds, Inventor of Linux

Linux is a widely used operating system family, alongside Windows and macOS.
Linux is open source and is used on servers, cloud systems, laptops, embedded devices, and phones.
Android, a mobile operating system, is based on Linux.

Ubuntu?

Figure: Logo of Ubuntu

Ubuntu is a beginner-friendly Linux distribution.
It is common in classrooms, cloud images, WSL, and personal machines, but most commands in this guide also work on other Linux distributions.

SSH Keys

For the class server, use key-based SSH access.

1. Generate a key pair once on your laptop
2. Send only the public key to the instructor or server admin
3. Keep the private key secret

Example

```
$ ssh-keygen -t ed25519
$ cat ~/.ssh/id_ed25519.pub
$ ssh yourid@class-server
```

Password login may be disabled from outside the lab network.

Jump Host

Some class sessions reach the class server through a jump host. The jump host is only a bridge; do not run your analysis there.

Example

```
$ ssh -J jump-user@jump-host yourid@class-server
```

In MobaXterm, VS Code, or another SSH client, set the class server as the target and the jump host as the SSH gateway or ProxyJump host.
Use the host, port, and account details provided in class. Do not post them publicly.

Where we start

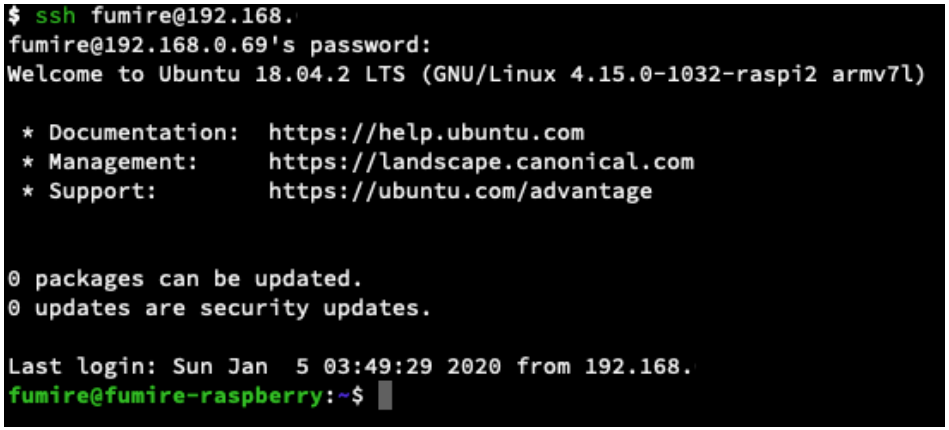

Figure: Here is where we start

After you open a terminal or connect to a server via SSH, you may see a prompt like this.
Here is where we start!
The text before '@' is the user name, and the text after '@' is the server or computer name.

pwd

```
fumire@fumire-raspberry:~$ pwd
/home/fumire
```

Figure: Result of *pwd* Command

pwd is abbr. of "Print Working Directory".
You can see where you are with *pwd* command.
Also, "/home/username" is your *home folder*, a.k.a. '∼'.

ls

```
fumire@fumire-raspberry:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  snap  Templates  Videos
```

Figure: Result of *ls* Command

ls stands for "List".
ls command lists current directory contents.
If current directory is empty, the result will be nothing.

Configuration

Before starting the lab, make a small working directory. No custom dotfiles, shell theme, or plugin setup is required.

Example

```
$ mkdir -p ~/linux-lab
$ cd ~/linux-lab
$ pwd
```

Note that you should input only the command after '$'.
On the class server, required tools are prepared by the admin. If a command is missing, ask the instructor.
On your own Ubuntu or WSL machine only, install basic tools with *sudo apt install vim curl wget screen*.

Configuration *(Cont.)*

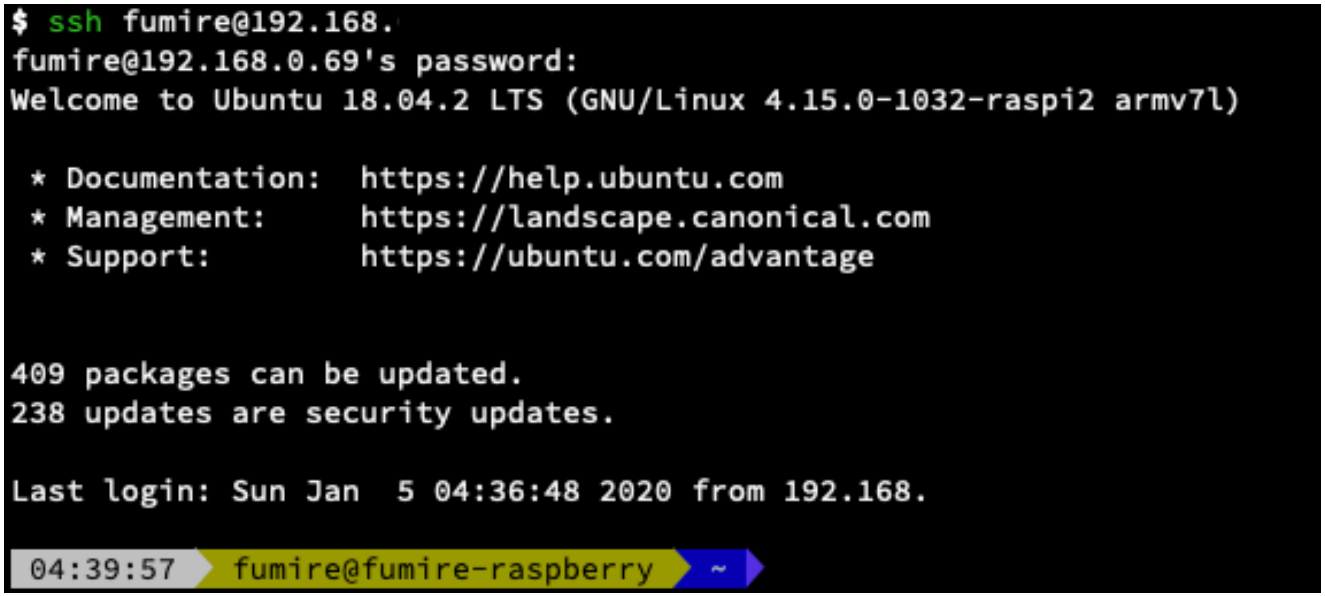

Figure: Shell Prompt

Your prompt may look different from this image. The important part is that you know which directory you are in.

Tip!

Figure: Right Command vs. Wrong Command

In this screenshot, valid commands are highlighted in green. Treat this as a helpful hint, not as a replacement for reading the command before pressing Enter.

mkdir

mkdir stands for "Make Directory".
You can make a directory which named 'test' as following:

Example

```
$ mkdir test
$ ls
```

mkdir returns nothing when it succeeds. Literally, *mkdir* command only makes a directory.
You can check that the directory has been made with *ls* command.

cd

cd is abbr. of "Change Directory".
You can change your working directory to 'test' as following:

Example

```
$ pwd
$ cd test
$ pwd
```

Also, you can go your home folder at once with *cd*, no matter where you are.

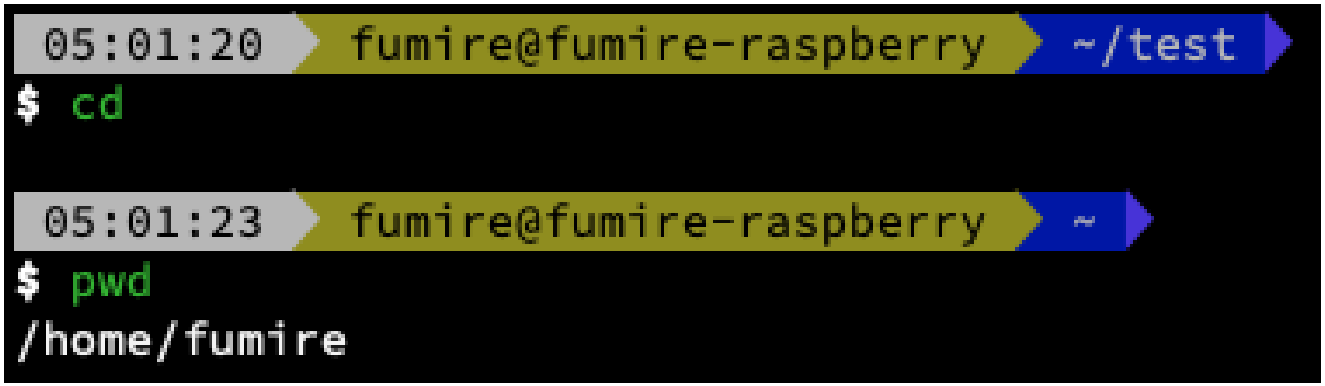

Figure: *cd* will guide you to home folder

Tip!

If you hit the "Tab" key, bash and zsh can complete file names, directory names, and commands.
Following example shows what tab completion gives.

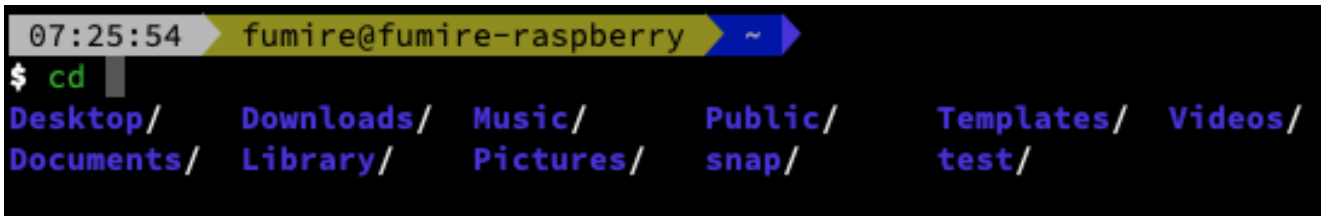

Figure: Shortcut with Tab

man and --help

You can get detailed information about command as following:

Example

$ man ls
and/or
$ ls --help

This guide gives simple information about Linux commands. When you want more detail about a command, use these commands.

Directory Structure

Try following commands:

Example

$ cd test
$ ls -al

Then, you can see like this:

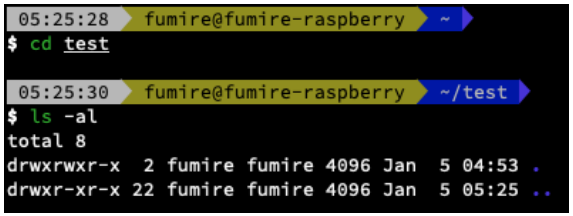

Figure: Result of *ls* command

All directory has '.' and '..', even though the directory is empty.
'.' means current directory itself; and, '..' means parent directory.

touch

touch command make new file or touch the file.
Try following example:

Example

```
$ cd
$ touch t
$ ls
```

Then, you can see that the file which name 't' has been made.

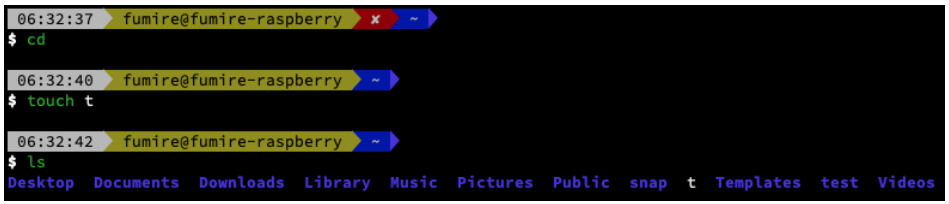

Figure: Result of *touch* Command

mv

mv command moves/renames file. *mv* is used as:

Example

$ mv SRC(source) DST(destination)

Try following commands:

Example

$ mv t tmp
$ ls
$ mv tmp test/
$ ls

Then, you will realize that the file 'tmp' is gone. I hope that you already know where the file goes. :)

cp

cp command copies SRC to DST. *cp* is used as:

Example

```
$ cp SRC DST
```

Try following commands:

Example

```
$ cd ~/test/
$ ls
$ cp tmp tmp2
$ ls
```

Then, you can realize that a new file 'tmp2' has been made.

rm

rm stands for 'Remove'. As its name, you can delete files or directories.

Example

```
$ rm tmp2
```

When you want to delete a directory, use '-r' option:

Example

```
$ rm -r directoryname
```

There is no way to restore removed files!! Beware what you remove!!

sudo

sudo runs a command with administrator privileges when the server policy allows it.
On the lab servers, ordinary users should not rely on *sudo*. Package installation, system settings, service restarts, and ownership changes are admin tasks.

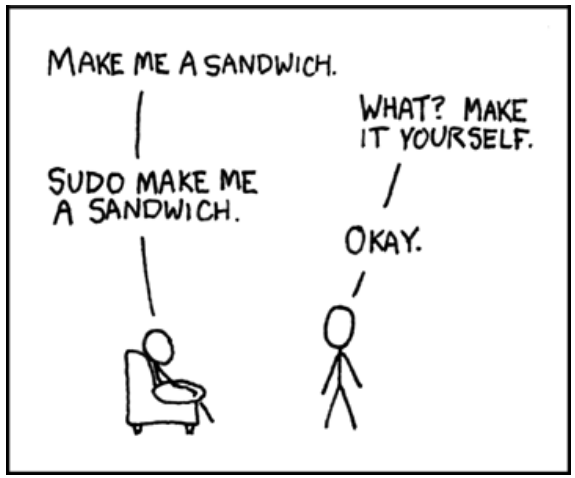

Figure: XKCD: Sandwich

If a command asks for *sudo*, stop and ask the instructor or server admin. Do not try to bypass the policy.

Editor

Common choices include nano, Vim/Neovim, Emacs, and VS Code Remote SSH.

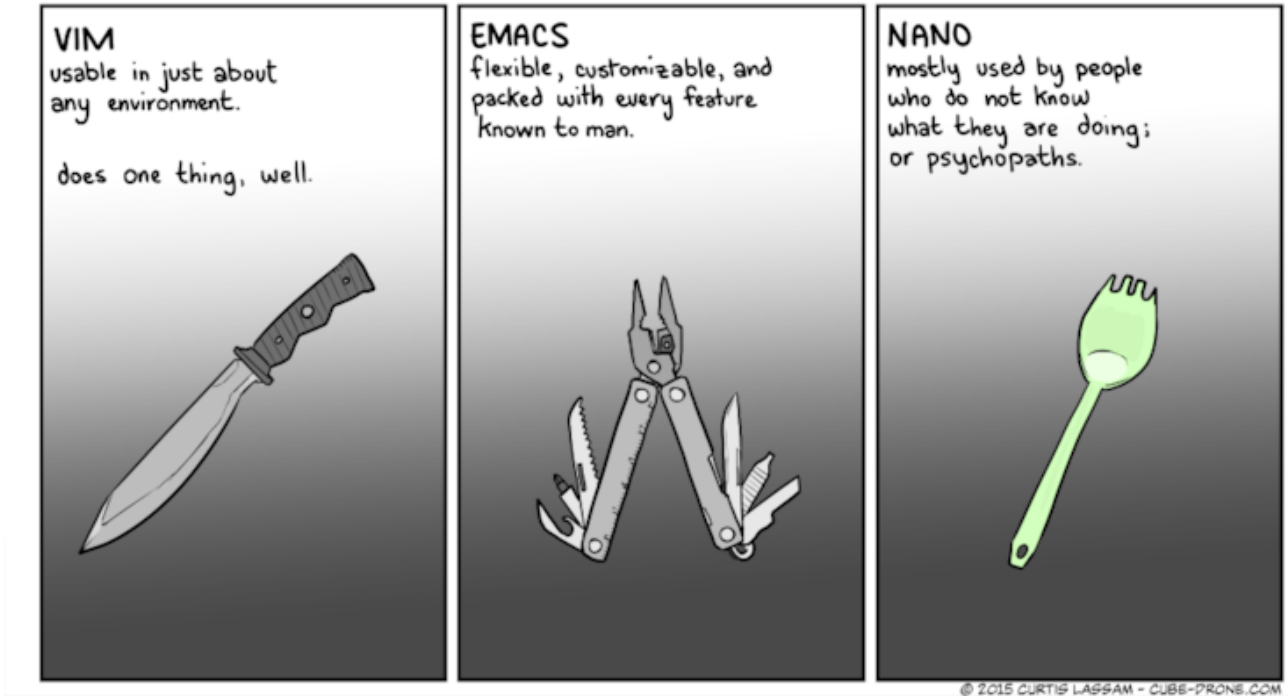

Figure: Descriptions of Editor

This guide uses Vim because it is available on many servers.

Editor *(Cont.)*

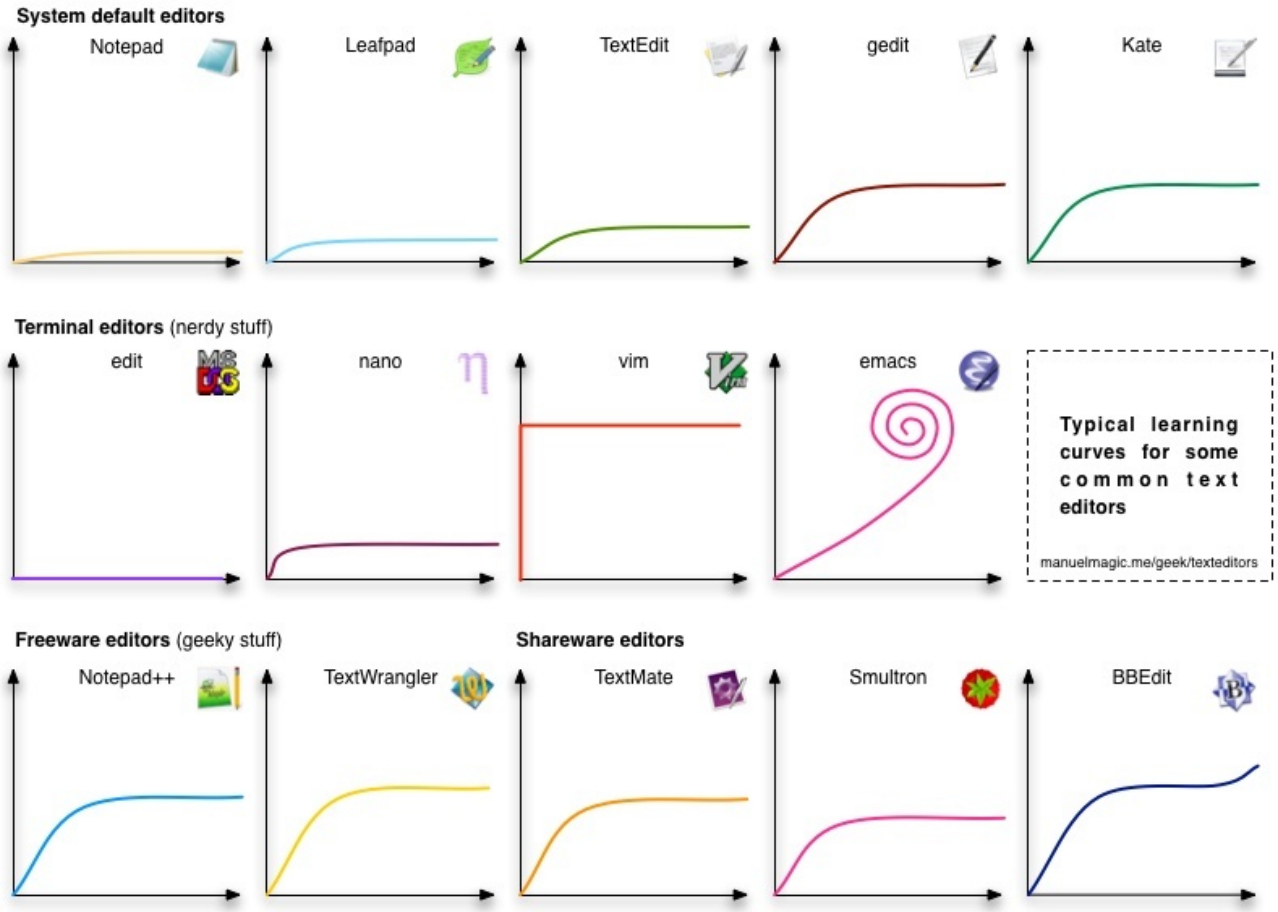

Figure: Learning Curves among Editors

First Meet with Vim

With these commands, you can make/edit file.

Example

```
$ vim notes.txt
```

If it is your first time to open Vim, then you may see something like this.

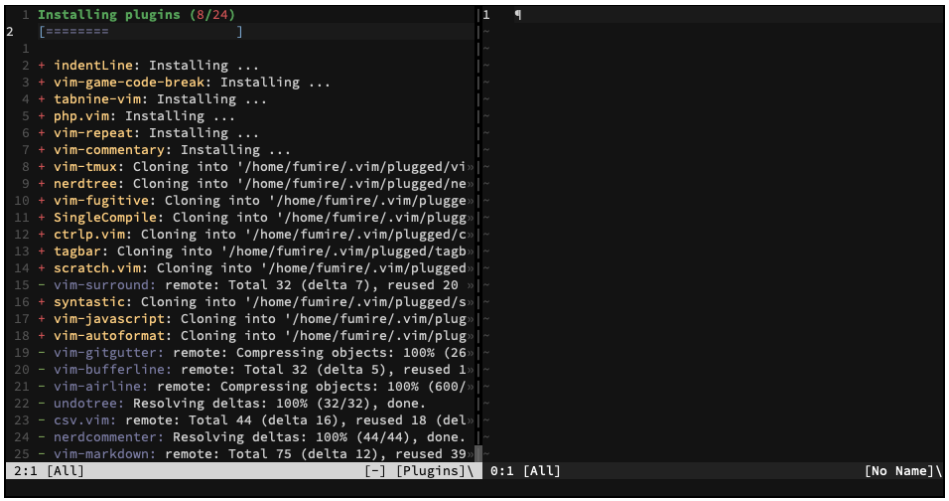

Figure: First Time of Vim

Modes of Vim

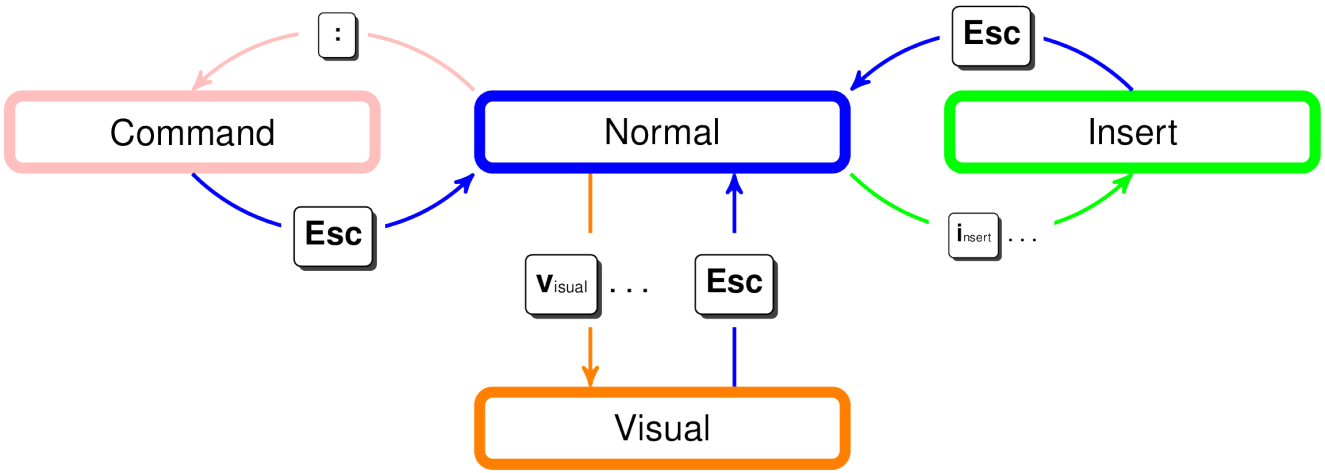

Figure: Three Modes in Vim

How to Edit with Vim

Editing with Vim follows these steps:

1. Press 'i'
2. Edit the file
3. Press 'ESC'
4. Enter ':w' which means *write*
5. Enter ':q' which means *quit*

Editor Practice (Optional)

No Vim plugin is required for this lab. If you want a guided practice session, run:

Example

```
$ vimtutor
```

If *vimtutor* is not installed on your own Ubuntu machine, install Vim with:

Example

```
$ sudo apt install vim
```

On the class server, ask the instructor instead of using *sudo*.

cat

cat stands for con**cat**enate. *cat* command reads files, and writing them to standard output.
Consider following example:

Example

```
$ cd ~/test/
$ cat tmp
```

You can see contents of file.

Input/Output to file

If you want redirect output to file, use following example:

Example

```
$ cat tmp > output
```

However, this method *overwrites* the contents of file.
If you want preserve the file contents, use following:

Example

```
$ cat tmp >> output
```

Also, $<$ means input from file.

Output to file *Cont.*

In some cases, you should divide standard output and standard error.
In these cases, use following commands:

Example

```
$ commands 1> STDOUT 2> STDERR
```

Example

```
$ cat tmp 1> ex.stdout 2> ex.stderr
```

If you want to save both standard output and standard error in one file:

Example

```
$ commands 1>log 2>&1
```

more / less

more and *less* are commands for seeing the contents of file. Consider following examples:

Example

```
$ more tmp
```

Example

```
$ less tmp
```

Also, hit "Q" when you want to leave *more* or *less*.

Pipe

Use pipe (|) to indicate input as output of previous command.

Example

```
$ command1 | command2
```

The output of command 1 will be the input of command2.
Consider following example:

Example

```
$ cat tmp | less
```

Two Ways for Download

There are two main ways for download.

1. curl
2. wget

curl

Example

```
$ curl https://example.com
```

curl returns to standard output. If you want to get file, consider following:

Example

```
$ curl -L https://example.com -o example.html
```

wget

wget returns a downloaded file as output.

Example

```
$ wget https://example.com -O example.html
$ ls
```

gzip

gzip is used for file compression and decompression.
When compressing:

Example

```
$ gzip tmp
```

When decompressing:

Example

```
$ gzip -d tmp.gz
```

However, the examples hereinabove delete the original files. If you want *keep* original file, consider following:

Example

```
$ gzip -k tmp
```

TAR Files

TAR stands for "Tape Archives".
Originally, it was used for tape; today, it is used to bundle many files into one archive.
TAR.GZ files are commonly used for source code and lab data. Replace LAB-URL with the URL from your instructor:

Example

```
$ wget LAB-URL -O lab-files.tar.gz
```

(TGZ is for TAR.GZ)

TAR Files *(Cont.)*

You might think you must decompress the GZ file and then extract the TAR file. However, you can extract a TGZ or TAR.GZ file at once:

Example

```
$ tar -xzf lab-files.tar.gz
```

Then, you can see the extracted directory with *ls*.

Permissions

With the following command, we can know how permissions are set:

Example

$ ls -al

```
08:16:10  fumire@fumire-raspberry  ~/test
$ ls -al
total 16
drwxrwxr-x  2 fumire fumire 4096 Jan  5 08:16 .
drwxr-xr-x 23 fumire fumire 4096 Jan  5 09:29 ..
-rw-rw-r--  1 fumire fumire    0 Jan  5 07:32 ex.stderr
-rw-rw-r--  1 fumire fumire    9 Jan  5 07:32 ex.stdout
-rw-rw-r--  1 fumire fumire    9 Jan  5 07:07 tmp
```

Figure: File Permissions

The way to read this result is following:

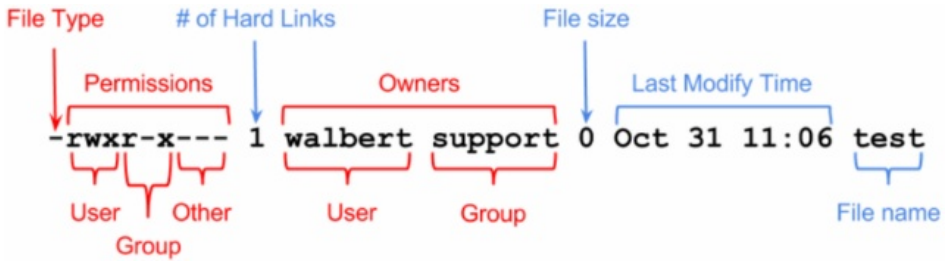

Figure: How to Read Permissions

chmod

chmod stands for "Change Mode". You can modify permissions of files. Before input command, you should calculate simple arithmetic:

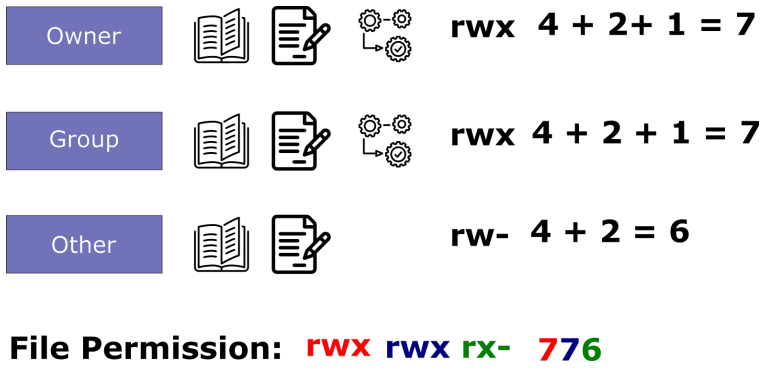

Figure: Simple Arithmetic

Then, you will get three digits for permission. Moreover, 660 or 770 are usually used.

Example

$ chmod three_digits filename

chmod *(Cont.)*

Or, you can do as followings:

Example

```
$ chmod rwx------ tmp
```

Example

```
$ chmod g=rwx tmp
```

Example

```
$ chmod +x tmp
```

chown

chown stands for "change ownership". As its name, you can modify ownership of file. On shared lab servers, changing file owner is usually an administrator task.
When you want to change only USER:

Example

```
$ chown USER tmp
```

When you want to change both USER and GROUP:

Example

```
$ chown USER:GROUP tmp
```

Ctrl-C

When a foreground command is running and you want to stop it, press "Ctrl-C".

Example

```
$ sleep 600
$ ^C
```

In terminal output, ^ means "Control"; ^C means Ctrl-C.
Ctrl-C sends SIGINT, which stands for "signal interrupt". Most command-line programs stop after receiving it.

&

Some commands take longer than a few seconds:

Example

```
$ sleep 600
```

Add '&' to start the command in the background and get your prompt back:

Example

```
$ sleep 600 &
```

The job is still running in this shell. Use *jobs* to see it.

jobs

jobs lists background jobs started from the current shell.

Example

$ jobs

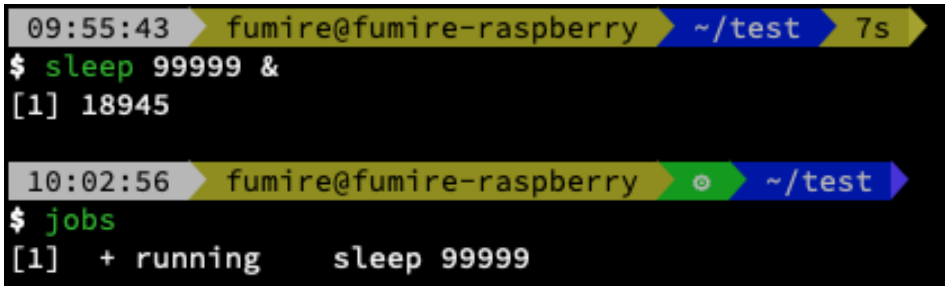

Figure: Result of *jobs*

The number in square brackets is the job number for this shell. Use it with commands such as *fg*, *bg*, or *kill*.
If you close the shell, this job list is gone.

kill

kill sends a signal to a process. By default, it asks the process to terminate cleanly.
To stop a background job from the current shell, use its job number:

Example

```
$ kill %1
```

The number comes from the *jobs* command.

Example

```
$ kill 12345
```

Use a PID when the process was not started from the current shell. Only stop processes that belong to you.

nohup

If SSH disconnects, the shell may send SIGHUP to commands it started.
A background command can stop even though you used '&'.
nohup tells one command to ignore SIGHUP.

Example

```
$ nohup sleep 600 > sleep.log 2>&1 &
```

Without redirection, *nohup* writes output to *nohup.out*. Use it for small, simple commands only.
For long analysis or shared CPU/GPU work, submit a Slurm job instead.

screen

screen keeps a terminal session running on the server after SSH disconnects.

Example

```
$ screen -S lab
Ctrl-a, then d
$ screen -r lab
```

Press Ctrl-a, then d inside *screen* to detach. It is not a shell command. Use *exit* when you are done. *screen* keeps the shell alive, but it does not reserve CPU, memory, or GPU. Use Slurm for computation.

Slurm

The lab server is shared. CPU, memory, GPU, storage, and time are distributed across all users.
Slurm is the job scheduler. It queues your work, starts it when the requested resources are available, and writes output to log files.
Use the shell for quick checks and file editing. Use Slurm for long runs, many cores, high memory, or GPU jobs.

Slurm Script

A Slurm script is a shell script with resource requests at the top. Create *hello-slurm.sh* with your editor, then add:

Example

```
#!/bin/bash
#SBATCH --job-name=linux-lab
#SBATCH --output=slurm-%j.out
#SBATCH --time=00:05:00
#SBATCH --mem=1G
#SBATCH --cpus-per-task=1
hostname
sleep 30
```

#SBATCH lines define the job name, log file, wall time, memory, and CPU request.

sbatch

Submit the script to the queue with *sbatch*:

Example

```
$ sbatch hello-slurm.sh
```

Slurm prints a job ID, such as "Submitted batch job 12345". Save that ID to track the job and find the output file:

Example

```
$ squeue -u $USER
$ cat slurm-JOBID.out
```

squeue

Check queued and running jobs with *squeue*:

Example

```
$ squeue -u $USER
```

Common states:

1. PD: waiting in the queue
2. R: running
3. CG: completing and writing output

A PD job is not broken; it is waiting for resources or priority. If Slurm does not respond, report it to the instructor or server admin.

scancel

Cancel a submitted job with *scancel*:

Example

```
$ scancel JOBID
```

Cancel all of your own queued or running jobs only when you really mean it:

Example

```
$ scancel -u $USER
```

The JOBID from *sbatch* is the handle for checking, reading logs, and canceling. Use the all-job cancel command carefully.